

The Solopreneur Ops System

PRINTMAXX | printmaxxer.gumroad.com

This product is for personal use only. Do not redistribute.

The Solopreneur Ops System: How to Run a One-Person Business That Never Sleeps

The exact operating system behind a solo operation managing 40+ revenue streams, 80+ automation scripts, and 1,600+ leads -- without a team.

What this is

This is the operating system I built to run my entire business. Not a philosophy. Not a framework you have to adapt. The actual files, scripts, schedules, and checklists I use every day.

I run 40+ revenue streams simultaneously. Apps, digital products, freelance services, content, cold outreach, affiliate programs. One person. No employees. The only way this works is with systems.

This document gives you those systems.

Part 1: The Three-Layer Architecture

Every solopreneur operation needs three layers running simultaneously. Miss one and the whole thing breaks.

Layer 1: Always-Running Automation (No Human Needed)

These run on a schedule regardless of whether you're awake, asleep, or on vacation.

What goes in Layer 1:

- Lead scrapers that pull fresh prospects daily
- Content schedulers that post across 6+ platforms
- Price monitors that track competitor changes
- Performance trackers that score your active ventures
- Revenue dashboards that update automatically

How to set it up:

Use cron jobs. A cron job is a scheduled task on your computer (Mac/Linux) or server that runs automatically.

```
# Example: run lead scraper every day at 6 AM
0 6 * * * python3 /path/to/lead_scraper.py

# Example: run content poster every day at 8 AM, 12 PM, 5 PM
0 8,12,17 * * * python3 /path/to/content_poster.py

# Example: run performance tracker every Monday at 3 AM
0 3 * * 1 python3 /path/to/performance_tracker.py
```

My actual cron schedule (16 jobs):

Time	What Runs	Purpose
2:00 AM	Master overnight runner	Executes 30+ scripts in sequence
6:00 AM	Twitter scraper	Monitors 92 accounts for new tact..
6:15 AM	Reddit scraper	Monitors 41 subreddits for opport..
6:30 AM	Content calendar updater	Prepares today's posts
7:00 AM	Lead scorer	Re-scores all leads with fresh data
8:30 AM	Daily TODO generator	Creates prioritized action list
Every 30 min (midnight-8 AM)	Auto-resume monitor	Restarts interrupted overnight sc..
Weekly Monday 3:00 AM	Platform RPM tracker	Tracks monetization rates across ..
Weekly Monday 3:30 AM	Creator program monitor	Checks for new creator fund oppor..
Weekly Monday 4:00 AM	ASO keyword research	Finds trending app keywords

The key insight: your overnight script should be idempotent. That means if it gets interrupted and restarts, it picks up where it left off without duplicating work. Check if output files already exist before creating them.

Layer 2: AI-Assisted Sessions (Your Primary Work)

This is when you sit down and work. But instead of doing everything from scratch, you pick up from where your automated systems left off.

Morning routine (15 minutes):

1. Check overnight log: `tail -50 logs/overnight_$(date +%Y-%m-%d).log`
2. Read auto-generated TODO: `cat DAILY_TODO_$(date +%Y_%m_%d).md`
3. Review new leads that came in overnight
4. Check which content posted and how it performed
5. Start executing from your prioritized TODO list

Your TODO generator should scan:

- Overnight scraper results (new data that needs human review)
- Pending alpha (tactics flagged for your review)
- New leads (prospects that need outreach)
- Content ready to publish (drafts awaiting approval)
- Revenue gaps (streams that haven't been updated)
- Deployment status (apps/products ready to launch)

Session structure (2-4 hours of focused work):

Block	Duration	What You Do
Morning review	15 min	Read overnight results, prioritize
Deep work block 1	90 min	Highest-value task from TODO
Quick wins	30 min	3-5 tasks that take <5 min each
Deep work block 2	60 min	Second-highest value task
End-of-session	15 min	Update trackers, queue overnight ..

Layer 3: Fallback Mode (When Everything Breaks)

Your AI tools will hit rate limits. Your scripts will fail. Your computer will crash at 3 AM mid-scrape. You need a fallback.

Fallback hierarchy:

1. Primary AI tool hits limits -> Switch to backup AI tool
2. Backup AI tool also limited -> Run pure Python scripts (no AI needed)
3. Scripts fail -> Manual checklist (pen and paper still works)

My three-layer fallback:

```
Layer 2 (AI session) fails
-> Layer 3a: Try backup AI tool with handoff state file
-> Layer 3b: Re-run Layer 1 scripts (pure Python, no AI)
-> Layer 3c: Generate manual TODO from latest data
```

The handoff state file is critical. When Layer 2 fails, it saves:

- Timestamp of failure
- Reason for failure
- What was completed before failure
- Next priorities for whoever picks up
- What NOT to do (safety rails)

```
{
  "timestamp": "2026-02-12T03:42:00",
  "reason": "token_limit",
  "last_action": "completed lead scoring, started outreach",
  "next_priorities": [
    "Send outreach to top 20 scored leads",
    "Post scheduled content to Buffer",
    "Review overnight scraper output"
  ],
  "do_not": [
    "Delete any files",
    "Make purchases",
    "Modify core scripts"
  ]
}
```

Part 2: The File System (Your Source of Truth)

Forget Notion. Forget Trello. Forget project management apps. Your source of truth is CSV files on disk.

Why:

- CSV files never break
- Every tool can read them

- They're version-controlled with git
- They load instantly (no API calls)
- They work offline
- AI tools read them natively

Directory structure

```
YOUR_BUSINESS/
  LEDGER/                # Source of truth. All tracking data.
    ALPHA_STAGING.csv    # New tactics/opportunities awaiting review
    REVENUE_STREAMS.csv  # All 40+ revenue streams tracked
    ACCOUNTS.csv         # Every platform account, status, warmup stage
    CONTENT_CALENDAR.csv # 30 days of scheduled content
    LEADS/               # Lead lists by industry/source

  AUTOMATIONS/          # All scripts that run automatically
    logs/               # Output from every script run
    leads/              # Scraped lead data
    content_posting/     # Buffer-ready CSV files

  OPS/                  # Operating procedures
    DAILY_TODO_*.md      # Auto-generated daily priorities
    SESSION_HANDOFF.md   # Current state for session continuity

  MONEY_METHODS/        # Playbook per revenue stream
    COLD_OUTBOUND/       # Email sequences, templates, deliverability
    APP_FACTORY/         # App specs, GTM plans, ASO research
    LOCAL_BIZ/           # Local business service templates
    CONTENT_FARM/        # Content strategies per niche

  FINANCIALS/           # Revenue, expenses, P&L
    REVENUE_TRACKER.csv  # Every dollar, where it came from
    EXPENSE_TRACKER.csv  # Every dollar spent
```

Tracking files you need (minimum viable ops)

1. Revenue Streams Tracker

```
stream_id,method,platform,status,monthly_revenue,last_updated,notes
RS001,gumroad_digital,Gumroad,ACTIVE,127,2026-02-10,3 products live
RS002,fiverr_services,Fiverr,SETUP,0,2026-02-08,2 gigs listed
RS003,cold_email_dental,Direct,ACTIVE,2500,2026-02-10,3 clients
```

Track every revenue stream. Even the ones making \$0. Especially the ones making \$0. You need to see which ones to kill and which to double down on.

2. Alpha Staging (Opportunity Pipeline)

```
alpha_id,date,source,tactic,category,status,roi_potential,notes
ALPHA001,2026-02-10,@handle,6-question cold email framework,OUTBOUND,APPROVED,HIGH,tested 14% reply rate
ALPHA002,2026-02-10,r/juststart,keyword clustering for programmatic SEO,SEO,PENDING_REVIEW,MEDIUM,needs
```

validation

Every tactic you discover goes here. Status flows: PENDING_REVIEW -> APPROVED -> INTEGRATED (or REJECTED).

3. Accounts Tracker

```
platform,username,email,status,warmup_day,followers,revenue,notes
Gumroad,printmaxxer,email@gmail.com,ACTIVE,n/a,0,127,3 products live
Fiverr,printmaxx_dev,email@gmail.com,WARMUP,14,0,0,2 gigs pending review
Twitter,@PRINTMAXXER,email@gmail.com,ACTIVE,45,1247,0,posting daily
```

Part 3: The Daily Cycle

Your business runs on a 24-hour cycle. Every hour has a purpose.

The Cycle

```
10 PM - Midnight: Queue overnight scripts, set up tomorrow's TODO inputs
Midnight - 8 AM:  Automated scripts run (scrapers, monitors, analyzers)
8:30 AM:          Daily TODO auto-generates from overnight results
9 AM - 12 PM:     Deep work session 1 (highest-value tasks)
12 PM - 1 PM:     Quick wins (respond to leads, approve content, check metrics)
1 PM - 3 PM:      Deep work session 2 (building, shipping, deploying)
3 PM - 4 PM:      Research session (scan alpha, review opportunities)
4 PM - 5 PM:      End of day (update trackers, plan tomorrow, queue scripts)
5 PM - 10 PM:     Off (or optional evening build session)
```

The daily TODO generator

This is the most important script in your entire operation. It replaces project management tools.

What it scans:

1. Overnight log files (what ran, what failed, what produced data)
2. Lead CSVs (how many new leads came in, their scores)
3. Alpha staging (how many opportunities need review)
4. Content calendar (what's scheduled for today)
5. Revenue tracker (what needs updating)
6. Account status (what's in warmup, what's ready to launch)

What it outputs:

```
# Daily TODO - 2026-02-12

## URGENT (Do First)
- [ ] 14 new dental leads (score 70+) need outreach emails
- [ ] Gumroad Product #4 launch blocked on cover image
```

```
- [ ] Fiverr gig review pending (submitted 3 days ago)

## HIGH PRIORITY
- [ ] Review 23 pending alpha entries from overnight scrape
- [ ] Post 4 scheduled tweets (Buffer CSV ready)
- [ ] Cold email campaign: 50 sends queued, need to approve

## STANDARD
- [ ] Update revenue tracker (Gumroad sales from yesterday)
- [ ] Check overnight scraper logs for errors
- [ ] Run lead scorer on new batch

## LOW PRIORITY
- [ ] Research 3 new app ideas from trending list
- [ ] Draft newsletter issue for next week
```

The TODO file is ephemeral. New one every day. Old ones get archived automatically.

Part 4: The Perpetual Improvement Loop

This is what separates a business that stagnates from one that compounds.

The Five Loops

Every business operation is one of five loops running simultaneously:

Loop 1: Research -> Alpha -> Integration

```
Scan sources (Twitter, Reddit, competitors)
-> Find tactics/opportunities
-> Score them (0-100 based on ROI, effort, risk)
-> Approve high-scorers
-> Integrate into your playbooks
-> Repeat daily
```

Loop 2: Content -> Post -> Measure -> Improve

```
Create content (tweets, articles, videos)
-> Schedule across platforms
-> Measure engagement (reply rate, not likes)
-> Double down on winners
-> Kill losers
-> Repeat daily
```

Loop 3: Product -> List -> Sell -> Track

```
Build product (digital, app, service)
-> List on marketplace (Gumroad, Etsy, Fiverr)
-> Drive traffic (content, cold email, affiliates)
-> Track revenue per product
-> Improve top sellers, kill bottom performers
-> Repeat weekly
```

Loop 4: Lead -> Score -> Outreach -> Close

```
Scrape leads (Google Maps, Apollo, directories)
-> Score them (0-100 on fit, budget, pain signals)
-> Send cold email sequence (4 emails over 14 days)
-> Book calls with respondents
-> Close deals
-> Repeat daily
```

Loop 5: App -> Build -> Deploy -> ASO -> Measure

```
Identify app opportunity (trending category, underserved niche)
-> Build MVP (PWA or native, 1-2 week sprint)
-> Deploy (Vercel/Netlify free tier)
-> Optimize for app stores (keywords, screenshots, description)
-> Measure installs and retention
-> Iterate or kill
-> Repeat monthly
```

Scoring system for ventures

Every venture in your portfolio gets a score. This prevents emotional attachment to losers.

```
SCORE = (Revenue * 0.3) + (Growth_Rate * 0.25) + (Time_Invested_Efficiency * 0.2) + (Automation_Level * 0.15)
+ (Market_Validation * 0.1)
```

Score	Action
80-100	Double down. Allocate more time a..
60-79	Maintain. Keep running, look for ..
40-59	Watch. Give it 30 more days, then..
20-39	Wind down. Stop active work, let ..
0-19	Kill. Archive everything, realloc..

Run this scoring monthly. Be ruthless. The ventures you kill free up resources for the ones that are working.

Part 5: The Session Handoff Protocol

This is how you maintain continuity across work sessions without losing context.

The problem

You stop working at 5 PM. You pick up again at 9 AM. That's 16 hours of gap. Without a handoff, you spend 30 minutes just remembering where you were.

The solution

Every session ends with a handoff file update. Takes 5 minutes. Saves 30 minutes tomorrow.

SESSION_HANDOFF.md template:

```
# Session Handoff - [DATE]

## What got done this session
- Deployed 3 Gumroad products (Cold Email Pack, Funnel Teardown, AI Blueprint)
- Sent 50 cold emails to dental practices in Austin TX
- Fixed broken overnight scraper (Reddit API changed endpoint)

## What's in progress
- Fiverr gig review pending (submitted today, typically 24-48h)
- Cold email warmup on Domain 2 (Day 14 of 21)
- App build: Ramadan tracker at 70% completion

## Blockers
- Need Stripe account to accept payments on Gumroad
- Vercel deployment requires `vercel login` (need to run in terminal)

## Tomorrow's priorities (human judgment, not auto-generated)
1. Check Fiverr gig approval status
2. Continue cold email sends (50/day target)
3. Finish Ramadan tracker and deploy

## Numbers
- Revenue today: $47 (2 Gumroad sales)
- Leads generated: 14 (dental scraper)
- Content posted: 6 tweets, 1 Medium article
- Cold emails sent: 50 (3 replies, 1 positive)
```

Git as memory

Every session, commit your changes. Your git history IS your business journal.

```
git add -A
git commit -m "Session 2026-02-12: deployed 3 products, sent 50 cold emails, fixed reddit scraper"
```

If something breaks, you can always roll back. If you need to review what you did last Tuesday, check the commit log.

Part 6: Revenue Tracking That Actually Works

The minimum viable financial system

Three files. That's it.

REVENUE_TRACKER.csv

```
date,method_id,method_name,platform,amount,type,notes
2026-02-10,RS001,cold_email_subject_lines,Gumroad,5.00,sale,organic search
```

```
2026-02-10,RS003,dental_website,Direct,2500.00,service,Austin TX practice
2026-02-11,RS001,cold_email_subject_lines,Gumroad,5.00,sale,twitter referral
```

EXPENSE_TRACKER.csv

```
date,category,item,amount,recurring,notes
2026-02-01,tools,Instantly.ai,97.00,monthly,cold email automation
2026-02-01,domains,3 cold email domains,36.00,annual,porkbun
2026-02-05,ads,Facebook test,100.00,one-time,testing dental ad
```

P_AND_L_MONTHLY.csv

```
month,revenue,expenses,profit,margin,top_method,notes
2026-01,0,133,negative_133,n/a,none,setup month
2026-02,2557,197,2360,92%,cold_email_dental,first client month
```

The weekly financial review (30 minutes)

Every Sunday:

1. Update REVENUE_TRACKER with any transactions you missed
2. Update EXPENSE_TRACKER
3. Calculate this week's P&L
4. Compare to last week
5. Score all ventures (see scoring system above)
6. Decide: what to double down on, what to kill

Part 7: Content as a Compound Machine

The multiplication principle

One piece of work becomes many pieces of content. This is non-negotiable.

Build an app

- > "How I built X" tweet thread
- > Tutorial blog post
- > Reddit post in r/SideProject
- > YouTube Shorts walkthrough (60 seconds)
- > Newsletter deep dive
- > Gumroad "How I Built This" product (\$9)
- > Affiliate links for every tool used

Research alpha

- > Tweet with one key insight (tease, don't give everything)
- > Full breakdown in newsletter (drive signups)
- > Gumroad mini-product if 10+ related insights (\$5)
- > Reddit value post (give 80%, gate 20%)

Close a deal

```
-> "How I closed $X from cold email" thread
-> Case study for your portfolio
-> Cold email template product (the exact emails that worked)
-> Testimonial request from client
```

The content funnel

Every piece of content serves one purpose: move the reader closer to paying you.

```
Free content (tweets, Reddit, articles)
-> Newsletter signup (Beehiiv, free tier)
-> Free product (Gumroad, $0, captures email)
    -> Paid product ($9-$29)
        -> Service ($500-$3,000)
            -> Retainer ($200-$500/month)
```

Every tweet ends with value, not a beg. No "follow me for more." Just deliver so much value they find your profile and follow on their own.

Part 8: Automation Recipes (Copy These)

Recipe 1: Daily lead scraper

```
# Pseudocode - adapt to your stack
def daily_lead_scraper():
    cities = load_csv("cities_top50.csv")
    industries = ["dentist", "lawyer", "realtor"]

    for city in cities[:5]: # 5 cities per day, rotate
        for industry in industries:
            leads = scrape_google_maps(city, industry)
            scored = score_leads(leads) # 0-100 based on website quality, reviews, etc
            save_to_csv(scored, f"leads/{city}_{industry}_{today}.csv")

    # Generate summary
    total = count_new_leads()
    log(f"Scraped {total} new leads across {len(cities)} cities")
```

Recipe 2: Content scheduler

```
def schedule_content():
    calendar = load_csv("CONTENT_CALENDAR.csv")
    today_posts = [row for row in calendar if row["date"] == today]

    for post in today_posts:
        if post["platform"] == "twitter":
            queue_to_buffer(post["content"], post["time"])
        elif post["platform"] == "medium":
```

```
create_draft(post["content"], post["title"])

log(f"Scheduled {len(today_posts)} posts for today")
```

Recipe 3: Revenue dashboard

```
def generate_dashboard():
    revenue = load_csv("REVENUE_TRACKER.csv")
    expenses = load_csv("EXPENSE_TRACKER.csv")

    this_month = filter_current_month(revenue)
    total_rev = sum(row["amount"] for row in this_month)
    total_exp = sum(row["amount"] for row in filter_current_month(expenses))

    by_method = group_by(this_month, "method_name")
    top_method = max(by_method, key=lambda m: sum(r["amount"] for r in by_method[m]))

    print(f"Revenue: ${total_rev}")
    print(f"Expenses: ${total_exp}")
    print(f"Profit: ${total_rev - total_exp}")
    print(f"Top method: {top_method}")
    print(f"Active streams: {len(set(r['method_id'] for r in this_month))}")
```

Recipe 4: Alpha scorer

```
def score_alpha(entry):
    score = 0

    # Has specific numbers? (+30)
    if has_numbers(entry["description"]):
        score += 30

    # Has proof (screenshot, case study, GitHub stars)? (+25)
    if entry["proof_type"] != "none":
        score += 25

    # Can implement this week? (+20)
    if entry["implementation_time"] <= 7:
        score += 20

    # Multiple sources confirm? (+15)
    if entry["confirmation_count"] >= 2:
        score += 15

    # High engagement on source? (+10)
    if entry["source_engagement"] >= 1000:
        score += 10

    return score
```

Part 9: The Overnight Protocol

Before bed checklist (10 minutes)

- ☐ Commit all changes to git
- ☐ Queue overnight scripts (or verify cron is active)
- ☐ Update SESSION_HANDOFF.md
- ☐ Check that content is scheduled for tomorrow morning
- ☐ Verify lead scraper has cities/industries queued
- ☐ Set tomorrow's deep work priority (write it down NOW, not tomorrow)

Morning checklist (15 minutes)

- ☐ Read auto-generated DAILY_TODO
- ☐ Check overnight log for errors or new data
- ☐ Review new leads (how many, what scores)
- ☐ Check revenue tracker (any sales overnight)
- ☐ Scan content performance (what posted, any engagement spikes)
- ☐ Start deep work block 1

The compound effect

Day 1: You set up the system. You scrape 50 leads. You write 3 tweets. You list 1 product.

Day 30: Your system has scraped 1,500 leads. Posted 90 tweets. Listed 10 products. Sent 1,000 cold emails. Booked 15 calls. Closed 3 deals.

Day 90: 4,500 leads. 270 tweets. 30 products. 3,000 cold emails. 45 calls. 10 deals. \$15K-30K cumulative revenue.

You did not work 90x harder. The system compounded.

Part 10: Scaling Decisions

When to add tools (not before)

Revenue	What to Add	Cost
\$0	Free tools only (cron, Python, fr..	\$0
\$500/mo	One paid tool (Instantly.ai OR Bu..	\$97/mo
\$2K/mo	Dedicated email warmup + CRM	\$200/mo
\$5K/mo	VA for repetitive tasks (5-10 hrs..	\$500/mo
\$10K/mo	Paid ads budget for proven funnels	\$1-2K/mo
\$20K/mo	Full-time VA or part-time special..	\$2-4K/mo

Rule: never spend more than 30% of revenue on tools and team until you're above \$10K/month. Below that, your

constraint is not tools. It's execution.

When to kill a venture

Kill if any of these are true after 30 days of active work:

- Zero revenue AND zero engagement AND zero leads
- You dread working on it (energy matters for solopreneurs)
- The market shifted and the opportunity closed
- A competitor launched something 10x better with funding

Do NOT kill if:

- Revenue is small but growing week over week
- Engagement is building even without revenue
- You haven't actually done the work (no excuses, do the work first)

When to double down

Double down if:

- Revenue growing 20%+ week over week
- Reply rate above 8% on cold email
- Organic traffic growing without paid ads
- Customers asking for more / referring others
- You can see a clear path from current revenue to 10x

What to Do Right Now

1. Create the directory structure from Part 2
2. Set up your first 3 CSV tracking files (revenue, expenses, accounts)
3. Write your first SESSION_HANDOFF.md
4. Set up 1 cron job (start with the daily TODO generator)
5. Build 1 lead scraper for your primary industry
6. Write 3 pieces of content from your first day of work
7. List 1 product on 1 marketplace
8. Send 10 cold emails

That's your first day. Tomorrow, the system runs overnight and tells you what to do next.

Built by PRINTMAXX. This system runs 40+ revenue streams with one person and zero employees. The full automation toolkit (80+ Python scripts, cron configs, CSV templates): printmaxxer.gumroad.com

Disclaimer: Results not typical. Individual results vary based on effort, market conditions, and other factors.